GitHub Copilot

Construire une app en 2h avec l'IA

Formation pratique · FlowRange

Ce qu'on va faire aujourd'hui

> D'ici 2h, vous aurez construit une vraie application web de gestion de projet — avec GitHub Copilot.

Plan de session

#	Segment	Durée
1	GitHub Copilot, c'est quoi ?	5-10 min
2	Présentation du pitch FlowRange	5-10 min
3	Specs to Waves	15 min
4	Specs techniques	15 min
5	Implémentation Wave 1	30 min
6	Tests & améliorations	30 min

L'objectif n'est pas d'apprendre à coder

C'est d'apprendre à déléguer le code à l'IA

GitHub Copilot

C'est quoi exactement ?

4 façons d'utiliser Copilot

1. Copilot inline

Complétion de code dans l'éditeur, en temps réel.

> Appuyez sur `Tab` pour accepter, `Esc` pour refuser.

2. Copilot Chat

Conversation libre — poser des questions, expliquer du code, générer des snippets.

3. Copilot Edit

Modifier plusieurs fichiers en une seule instruction.

4. Copilot Agent

Mode autonome — il lit le projet, enchaîne les actions, écrit des fichiers entiers.

> C'est ça le **vibe coding**.

Les raccourcis essentiels

Raccourci	Action
`Ctrl+I`	Copilot inline edit (dans l'éditeur)
`Ctrl+Alt+I`	Ouvrir Copilot Chat
`Tab`	Accepter une suggestion inline
`Esc`	Refuser une suggestion inline
`Alt+]`	Voir la suggestion suivante

Les modes Copilot en résumé

```
Vous écrivez du code          →  Copilot inline complète
Vous posez une question       →  Copilot Chat répond
Vous demandez une modif       →  Copilot Edit modifie
Vous décrivez une feature     →  Copilot Agent construit
```

Aller plus loin

Les fonctionnalités avancées de Copilot

Vue d'ensemble

Fonctionnalité	Rôle
`copilot-instructions.md`	Instructions globales au niveau du repo
`.instructions.md`	Instructions scopées par dossier / type de fichier
`.prompt.md`	Prompts réutilisables dans le Chat
`SKILL.md`	Workflows spécialisés packagés dans `.github/skills/`
Agents	Modes personnalisés avec outils et instructions dédiés
MCP	Connecter Copilot à des outils et données externes
CLI	Copilot directement dans le terminal

Instructions — `copilot-instructions.md`

Fichier placé à la racine ou dans `.github/` — lu automatiquement par Copilot pour **tout le projet**.

```
# Mon projet

- Stack : Node.js 22, React 19, PostgreSQL
- Conventions : ESLint + Prettier, tests Jest
- Pas de `any` en TypeScript
- Toujours retourner des erreurs typées, jamais des strings
```

> **Portée** : l'ensemble du workspace.

Instructions scopées — `.instructions.md`

Des fichiers d'instructions **ciblés** sur un dossier ou un type de fichier, via un frontmatter `applyTo`.

```
---
applyTo: "**/*.test.ts"
---

# Conventions de tests

- Utiliser `describe` / `it` (pas `test`)
- Un seul `expect` par test
- Nommer les fichiers `*.spec.ts` pour les tests unitaires
```

> Placez ces fichiers dans le dossier concerné.
> Copilot les applique **uniquement** sur les fichiers correspondants.

Prompts réutilisables — `.prompt.md`

Des prompts complexes sauvegardés et appelables dans le Chat avec `#`.

```
---
description: Génère un CRUD complet pour une entité
---

Génère un CRUD complet pour l'entité décrite ci-dessous.
Respecte les conventions du projet (copilot-instructions.md).
Inclure : modèle, repository, service, controller, tests unitaires.

Entité : {{entité}}
```

Utilisation dans le Chat :

```
#generate-crud  Entité : Product
```

> Idéal pour les patterns répétitifs dans un projet.

Skills — `SKILL.md`

Un **skill** est un workflow spécialisé packagé avec ses best practices.

```
.github/skills/
  mon-skill/
    SKILL.md        ← instructions + workflow détaillé
```

Copilot charge le skill quand la tâche correspond à son domaine.

Exemples de skills utiles :

- Générer des tests de performance
- Créer un schéma de base de données à partir d'un modèle
- Scaffolder une API REST complète selon vos standards internes

> Partagez vos skills entre équipes pour standardiser les pratiques IA.

Agents — `.github/agents/`

Un **agent** est un mode Copilot personnalisé avec ses propres instructions, outils autorisés et comportement.

```
---
description: Expert en revue de code sécurité
tools:
  - codebase
  - githubRepo
---

Tu es un expert sécurité. Pour chaque fichier analysé, identifie
les vulnérabilités OWASP Top 10, propose des corrections concrètes
et explique le risque associé.
```

Cas d'usage :

- Agent dédié à la revue de sécurité

MCP

Model Context Protocol

Qu'est-ce que MCP ?

MCP (Model Context Protocol) est un standard ouvert qui permet à Copilot de se **connecter à des outils et données externes**.

Sans MCP → Copilot ne connaît que votre code ouvert dans VS Code.

Avec MCP → Copilot peut interagir avec :

- Vos bases de données
- Vos APIs internes
- GitHub, Azure, Jira, Figma...
- Le système de fichiers, Docker, Kubernetes...

> Think of it as **des plugins** pour votre agent IA.

MCP — Comment ça marche ?

```
// .vscode/mcp.json
{
  "servers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": { "GITHUB_TOKEN": "${env:GITHUB_TOKEN}" }
    },
    "postgres": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres",
               "postgresql://localhost/mydb"]
    }
  }
}
```

> Copilot découvre automatiquement les **outils** exposés par chaque serveur MCP.

MCP — Exemples concrets

Serveur MCP	Ce que Copilot peut faire
GitHub MCP	Lire les issues, créer des PRs, explorer les repos
Azure MCP	Interroger les ressources Azure, déployer
Filesystem MCP	Lire/écrire des fichiers hors du workspace
PostgreSQL MCP	Interroger la BDD, générer des migrations
Figma MCP	Lire les designs et générer les composants
Brave Search MCP	Rechercher sur le web en temps réel

Exemple de prompt avec MCP GitHub activé :

```
Crée une issue GitHub pour chaque bug identifié dans ce fichier de logs
```

MCP — En pratique

Activer un serveur MCP dans VS Code

1. Ouvrir la palette de commandes → "*MCP: Add Server*"
2. Ou éditer `.vscode/mcp.json` directement
3. Les outils apparaissent automatiquement dans Copilot Agent

Ressources :

- `modelcontextprotocol.io` — liste des serveurs disponibles
- `github.com/microsoft/mcp` — serveurs Microsoft officiels

> MCP transforme Copilot en un agent qui **agit** dans votre système, pas juste dans votre éditeur.

GitHub Copilot CLI

L'IA directement dans votre terminal

Copilot CLI — Installation

```
# Windows (PowerShell)
winget install GitHub.Copilot

# macOS / Linux
brew install copilot-cli
# ou : curl -fsSL https://gh.io/copilot-install | bash

# Lancer une session interactive
copilot
```

> Remplace l'ancienne extension `gh copilot` (deprecated oct. 2025).
> C'est un **agent complet** dans le terminal — pas juste suggest/explain.

Copilot CLI — Interface interactive

Lancez `copilot` dans un dossier de code et conversez en langage naturel :

```
> Montre-moi les commits de cette semaine et résume-les

> Trouve tous les fichiers modifiés il y a moins de 24h

> Crée une branche, corrige le bug #42, et ouvre une PR
```

Deux modes (basculer avec `Shift+Tab`) :

Mode	Comportement
Ask / Execute	Copilot propose des actions, vous validez une par une
Plan	Copilot construit un plan structuré avant d'agir

> Copilot demande votre approbation avant chaque action destructrice.

Copilot CLI — Usage programmatique

Pour des tâches ponctuelles sans entrer en mode interactif :

```
copilot -p "Revert the last commit" --allow-tool='shell(git)'
```

```
copilot -p "Liste les issues ouvertes qui me sont assignées"
```

Options de contrôle :

Option	Effet
`--allow-tool='shell(git)'`	Autorise uniquement les commandes git
`--deny-tool='shell(rm)'`	Interdit les commandes rm
`--allow-all-tools`	Autorise tout (attention !)

> Idéal pour intégrer Copilot dans des scripts ou des workflows CI.

Copilot CLI — Fonctionnalités clés

Ce que l'agent peut faire :

- Lire et modifier du code dans votre projet
- Exécuter des commandes shell (avec votre accord)
- Interagir avec GitHub : issues, PRs, Actions
- Utiliser des serveurs MCP configurés
- Gérer automatiquement le contexte (sessions quasi-infinies)

Commandes slash utiles :

Commande	Action
`/model`	Changer de modèle (Claude Sonnet 4.5, GPT-5…)
`/compact`	Compresser le contexte manuellement

FlowRange

"From chaos to flow"

Le problème

Les équipes jonglent entre **trop d'outils** :

- Un outil pour les tâches
- Un autre pour l'organisation visuelle
- Un troisième pour le suivi du temps

Résultat : données éparpillées, contextes perdus, heures perdues à synchroniser des tableaux.

La vision FlowRange

FlowRange réunit ce que vous aimez dans vos outils actuels :

Ce qu'on prend	Depuis quel outil
Simplicité visuelle, kanban, drag & drop	Trello
Statuts, vues multiples, assignations	Monday
Chronomètre par tâche, rapports automatiques	Toggl

> Une app tout-en-un pour **voir, organiser et mesurer** votre travail.

Les utilisateurs cibles

- 👨‍💼 **Consultants techniques** — suivi multi-missions, temps passé, vue d'ensemble
- 📋 **Chefs de projet** — vue consolidée en temps réel
- 📊 **Practice managers** — charge, vélocité, bottlenecks
- 💼 **Sales** — avancement des opportunités et avant-ventes

Ce que FlowRange permet

- Créer des **projets** organisés en **boards kanban**
- Gérer des tâches (titre, description, statut, priorité, assignation)
- **Déplacer** les tâches entre colonnes en glisser-déposer
- **Démarrer un chronomètre** sur une tâche
- Consulter un **dashboard** consolidé
- Gérer une **liste de membres**

Ce que FlowRange n'est PAS

> Définir les limites est aussi important que définir les features.

- ❌ Pas de notifications ni d'emails
- ❌ Pas d'intégrations tierces (Jira, Slack…)
- ❌ Pas de mode hors-ligne
- ❌ Pas de gestion des droits avancée

Ce pitch est volontairement incomplet

> C'est fait exprès. L'ambiguïté est le point de départ.

L'exercice consiste à utiliser GitHub Copilot pour **clarifier, questionner et compléter** ces specs — comme un vrai PO le ferait avec son équipe.

Questions à poser à Copilot :

- "*Qu'est-ce qui manque dans ce pitch pour pouvoir coder ?*"
- "*Quels cas limites n'ont pas été couverts ?*"
- "*Quelles ambiguïtés pourraient bloquer un développeur ?*"

Exercice

Specs to Waves

Qu'est-ce qu'une Wave ?

Une **wave** est un mini sprint autonome et livrable.

Règles d'une bonne wave :

- ✅ Livrable seule (pas de dépendance future)
- ✅ Visuellement perceptible par un utilisateur
- ✅ Réalisable en 2-4h de vibe coding
- ✅ Nommée avec un nom évocateur

Exercice — Specs to Waves (15 min)

Étape 1 — Créer `specs.md` et coller le pitch dedans

Étape 2 — Dans Copilot Chat (mode Agent) :

```
Tu es un product owner expert. À partir de ce pitch, génère des specs
fonctionnelles détaillées pour FlowRange. Pour chaque feature, décris
le comportement attendu, les cas nominaux et les cas limites.
Puis découpe ces specs en waves (mini sprints autonomes et livrables).
```

Étape 3 — Challengez le résultat avec les prompts de `waves-reference.md`

Les 5 Waves de FlowRange

Wave	Nom	Contenu
1	**The Board**	Kanban, drag & drop, CRUD tâches
2	**The Dashboard**	Stats, graphiques, synthèse
3	**The Timer**	Chronomètre, sessions de travail
4	**The Team**	Membres, assignations
5	**The Projects**	Multi-projets, multi-boards

> Aujourd'hui, on implémente la **Wave 1**.

Specs techniques

`copilot-instructions.md`

Votre contrat avec l'IA

Le fichier `copilot-instructions.md` dit à Copilot :

- **Ce qu'on veut** — stack, conventions, nommage
- **Ce qu'on ne veut pas** — pas d'auth, pas de libs exotiques
- **Le contexte** — structure de dossiers, modèle de données

> Plus il est précis, moins vous passerez de temps à corriger le code généré.

> Tout ce qui n'est pas précisé sera décidé par Copilot — stack, conventions, structure, choix d'architecture. Il comblera les blancs à sa façon, pas à la vôtre.

Ne faites pas confiance aveuglément

Copilot génère vite. Mais **vite ne veut pas dire juste**.

Ce qu'il faut toujours vérifier :

- Les versions des dépendances — *est-ce la dernière LTS ? Est-ce compatible ?*
- Les imports et packages utilisés — *existent-ils vraiment ?*
- La logique métier — *Copilot ne connaît pas vos règles implicites*
- La sécurité — *CORS, validation des inputs, secrets en dur...*

> 👁️ **L'exemple de cette session** : la stack mentionne `.NET 9` — avez-vous vérifié si c'est la bonne version ? C'est votre job de relire, pas celui de l'IA.

La règle d'or : Copilot génère, mais vous signez. Vous êtes **responsable de tout ce que vous acceptez** — code, architecture, instructions, ou specs.

Wave 1 — The Board

Implémentation live

Wave 1 — Objectif

> À la fin de cette wave, un utilisateur peut créer des tâches et les organiser sur un board kanban.

Features

- Board kanban — 3 colonnes : To Do, In Progress, Done
- Créer / afficher / supprimer des tâches
- **Drag & drop** entre colonnes
- Données persistées en mémoire

T+00 — Scaffolding (5 min)

```
# Backend
dotnet new web -n FlowRange.Api
cd FlowRange.Api

# Frontend
npm create vue@latest flowrange-frontend
# Choisir : TypeScript ✓, Vue Router ✓, Pinia ✓
```

T+05 — Générer le backend (8 min)

Dans Copilot Chat (mode Agent) depuis `FlowRange.Api` :

```
En te basant sur specs.md (Wave 1), génère l'implémentation complète côté backend :
- Le modèle Task avec les propriétés définies dans les specs
- Le FlowRangeDbContext en InMemory
- Les endpoints Minimal API : GET /tasks, POST /tasks,
  PATCH /tasks/{id}/status, DELETE /tasks/{id}
- La configuration CORS pour localhost:5173
- Des données de seed pour avoir 3-4 tâches au démarrage
```

T+13 — Générer le frontend (10 min)

```
En te basant sur specs.md (Wave 1), génère les composants Vue.js côté frontend :
- KanbanBoard.vue : affiche 3 colonnes (To Do, In Progress, Done)
- KanbanColumn.vue : affiche les cartes de son statut, accepte le drop
- TaskCard.vue : affiche titre, priorité, bouton supprimer
- Drag & drop natif HTML5 entre les colonnes
- Store Pinia pour gérer l'état des tâches
- Appels API dans services/api.ts
```

T+23 — Corrections & ajustements (7 min)

Problèmes courants et prompts Copilot pour les corriger :

Problème	Prompt
Erreur CORS	"*Le backend retourne une erreur CORS, corrige la config*"
URL API incorrecte	"*L'appel API échoue, l'URL pointe vers le mauvais port*"
UI ne se met pas à jour	"*Le composant ne se rafraîchit pas après le drag & drop, corrige ça*"

Wave 1 — Résultat attendu

Un board kanban fonctionnel avec :

- ✅ 3 colonnes avec cartes colorées
- ✅ Création de tâches via formulaire
- ✅ Drag & drop entre colonnes
- ✅ Suppression de tâche
- ✅ Indicateur de priorité

À vous de jouer

Tests & améliorations (30 min)

Continuez à explorer

Niveau standard

- Ajouter un indicateur de priorité coloré sur les cartes
- Enrichir le formulaire de création de tâches
- Améliorer le style (couleurs, animations)

Challenge senior

- "*Refactorise le store Pinia pour gérer les erreurs réseau*"
- "*Commence la Wave 2 : génère les stats pour le dashboard*"
- "*Ajoute de la validation côté backend et côté formulaire*"

Ce qu'on a appris aujourd'hui

1. **Copilot Agent** peut écrire une app entière à partir d'un brief
2. **Un bon prompt** vaut mieux que 100 lignes de code manuel
3. `copilot-instructions.md` est votre contrat avec l'IA
4. Définir ce qu'on **ne veut pas** est aussi important que ce qu'on veut
5. Les **waves** permettent de livrer de la valeur itérativement

Merci

> *"The best code is the code you didn't have to write."*

FlowRange · GitHub Copilot Formation · 2026